FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 4

Name: Aufwand in h:

Mat.Nr: Punkte:

Übungsgruppe: korrigiert:

Beispiel 1 (24 Punkte) Bilderverwaltung: Erstellen Sie eine Software zur Erzeugung und Verwaltung von Bildern mit graphischen Objekten. Die Bilder sind in einer Bilderverwaltung abgelegt und können durch generieren einer SVG-Datei in einem entsprechenden Browser angezeigt werden.

Eine Methode `WriteSVG` soll das Bild als SVG-Datei (*scalable vector graphics*) mit dem gegebenen Namen abspeichern. Der Name der SVG-Datei wird bei der Konstruktion eines Bildes festgelegt. Ein Beispiel für eine solche SVG-Datei könnte z.B. folgendermaßen aussehen:

```
<?xml version="1.0"?>
<svg xmlns="http://www.w3.org/2000/svg">
<rect x="10" y="10" width="90" height="70" stroke="black" fill="none"/>
<circle cx="55" cy="45" r="35" stroke="black" fill="none"/>
</svg>
```

Die SVG-Datei beschreibt die Bestandteile der Grafik (im obigen Beispiel ein Rechteck und ein Kreis). Stroke (`stroke="black"`) gibt die Farbe an und `fill` die Füllung. Wird `fill="none"` weggelassen, so wird das Objekt mit der angegebenen Farbe gefüllt.

Es sollen gefüllte und nicht gefüllte Kreise und Rechtecke erzeugt werden, wobei ein Bild nur gefüllte oder nur nicht gefüllte Objekte haben kann.

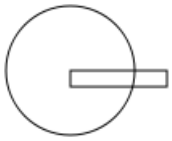


Bild1:test1.svg



Bild2: test2.svg

Die Bilderverwaltung und die Fabriken zur Objekterzeugung exisitieren nur einmal und werden beim

Beenden des Programmes automatisch freigegeben. Verwenden Sie zur Implementierung eine vorgestellte Methode aus der Vorlesung. Graphische Objekte werden mit einer entsprechenden Fabrik erzeugt und das Kopieren und Zuweisen soll verhindert werden!

Die Bilderverwaltung kann mit verschiedenen Fabriken arbeiten und so verschiedene Bilder erzeugen. Die Fabriken in der Bilderverwaltung sollen austauschbar sein!

Das Design soll so ausgelegt sein, dass in Zukunft nicht nur gefüllte und nicht gefüllte graphische Objekte erzeugt werden können, sondern auch z.B. schraffierte oder gepunktete graphische Objekte.

Für die SVG-Datei muss keine Grammatik angegeben werden!

Entwerfen Sie ein Klassendiagramm und verwenden Sie dazu entsprechende Design Pattern. Implementieren Sie das vollständige Klassendiagramm und schreiben sie einen Testtreiber. Verfassen Sie weiters eine Systemdokumentation (Funktionalität, Klassendiagramm, Schnittstellen der beteiligten Klassen, etc.)! Geben Sie die vom Testtreiber erzeugten SVG-Dateien sowohl textuell als auch graphisch ab.

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!

HSD
OBERÖSTERREICH
FH-HAGENBERG

Systemdokumentation

Player-Schnittstelle

Version 1.0

J. Rieger, L. Kieweg

Hagenberg, 28. Oktober 2026

Dieses Dokument zeigt den prinzipiellen Aufbau einer Systemdokumentation für Software-Projekte. Die einzelnen Kapitel sind mit Kommentaren versehen, welche die Struktur und den Inhalt erläutern.

Inhaltsverzeichnis

1 Organisatorisches **2**
1.1 Team . . . 2
1.2 Aufteilung der Verantwortungsbereiche . . . 2
1.3 Aufwand . . . 2

2 Anforderungsdefinition (Systemspezifikation) **3**
2.1 Nichtfunktionale Anforderungen . . . 3

3 Systementwurf **5**
3.1 Klassendiagramm . . . 5
3.2 Designentscheidungen . . . 6

4 Komponentenentwurf **6**

5 Testprotokollierung **7**
5.1 Ausgabe Konsole Testtreiber . . . 7
5.2 Erstellte Bilder . . . 7

6 Quellcode **9**

1 Organisatorisches

1.1 Team

- Julian Rieger, Matr.-Nr: S2510306023, E-Mail: S2510306023@fhooe.at
- Leander Kieweg, Matr.-Nr: S2510306014, E-Mail: S2510306014@fhooe.at

1.2 Aufteilung der Verantwortungsbereiche

- Julian Rieger
  - Implementierung der Klassen: Image, Object, GraphicObject, Circle, Rectangle, FillCircle, BlankCircle, FillRectangle, BlankRectangle
  - Erstellen des Klassendiagramms
  - Verfassen der Systemdokumentation
- Leander Kieweg
  - Implementierung der Klassen: GraphicObjectFactory, FillGraphicObjectFactory, BlankGraphicObjectFactory, GraphicManagement
  - Implementierung des Testtreibers

1.3 Aufwand

- Julian Rieger: geschätzt: 10h, tatsächlich 9h
- Leander Kieweg: geschätzt 10h, tatsächlich 9h

2 Anforderungsdefinition (Systemspezifikation)

Es wird eine Software zur Erzeugung und Verwaltung von Bildern mit grafischen Objekten erstellt. Ein Bild besteht aus gefüllten oder nicht gefüllten Kreisen und Rechtecken und kann als SVG-Datei gespeichert und in einem Browser angezeigt werden. Ein Bild enthält entweder nur gefüllte oder nur nicht gefüllte Objekte. Der Name der SVG-Datei wird bei der Konstruktion des Bildes festgelegt.

Funktionale Anforderungen

- `AddImage(fact, file)`: Legt ein neues Bild mit dem angegebenen Dateinamen an. Die übergebene Fabrik wird zur aktuellen Fabrik der Bilderverwaltung und erzeugt die Objekte dieses Bildes.
- `CreateCircle(stroke, x, y, radius)`: Erzeugt mit der aktuellen Fabrik einen Kreis (Mittelpunkt (x, y)) und fügt ihn dem zuletzt angelegten Bild hinzu.
- `CreateRectangle(stroke, x, y, width, height)`: Erzeugt mit der aktuellen Fabrik ein Rechteck (linke obere Ecke (x, y)) und fügt es dem zuletzt angelegten Bild hinzu.
- `WriteSVG()`: Schreibt alle Bilder der Bilderverwaltung als SVG-Datei. Ein einzelnes Bild kann ebenfalls über `Image::WriteSVG()` gespeichert werden.
- `ToSVG()`: Jedes grafische Objekt liefert sein SVG-Element als Text. Nicht gefüllte Objekte erhalten das Attribut `fill=nnone"`, bei gefüllten Objekten wird es weggelassen, sodass das Objekt mit der Strichfarbe gefüllt wird.

2.1 Nichtfunktionale Anforderungen

- Bilderverwaltung und Fabriken existieren nur einmal (Singleton) und werden beim Programmende automatisch freigegeben.
- Grafische Objekte werden ausschließlich über eine Fabrik erzeugt. Kopieren und Zuweisen von Objekten ist verboten.
- Die Fabriken der Bilderverwaltung sind austauschbar, sodass verschiedene Bilder mit verschiedenen Fabriken erzeugt werden können.
- Das Design ist erweiterbar: Weitere Objektfamilien (z.B. schraffierte oder gepunktete Objekte) können ergänzt werden, ohne bestehende Klassen zu verändern.
- Für die SVG-Datei muss keine Grammatik angegeben werden.

Fehlerbehandlung

Fehler werden als Ausnahmen (`std::exception`) gemeldet und vom Testtreiber abgefangen und per `std::cerr` ausgegeben:

- `std::logic_error`, wenn `CreateCircle` oder `CreateRectangle` aufgerufen wird, ohne dass eine Fabrik gesetzt oder ein Bild angelegt wurde.
- `std::runtime_error`, wenn eine SVG-Datei nicht geöffnet werden kann.

3 Systementwurf

3.1 Klassendiagramm

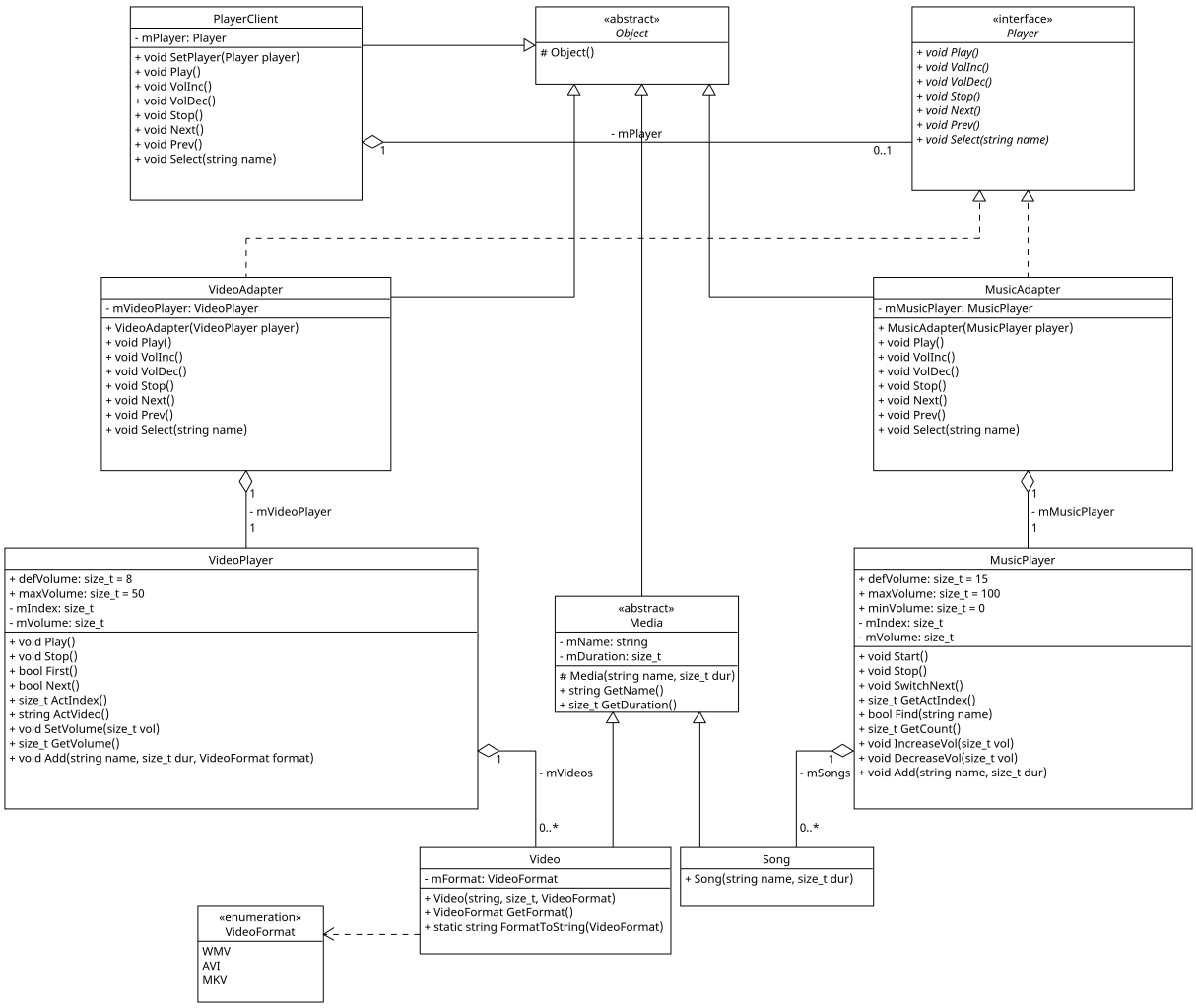


Abbildung 1: Klassendiagramm Bilderverwaltung

Anmerkung: Das Klassendiagram steht für bessere Lesbarkeit ebenfalls im Sysdoku-Ordner zur Verfügung.

3.2 Designentscheidungen

Für die Bilderverwaltung wurden folgende Entwurfsmuster eingesetzt, um Erweiterbarkeit und lose Kopplung zu gewährleisten.

Abstract-Factory-Pattern Die Schnittstelle `IGraphicObjectFactory` beschreibt die Erzeugung einer Familie zusammengehöriger Objekte (Kreis und Rechteck). Die konkreten Fabriken `BlankGraphicObjectFactory` und `FillGraphicObjectFactory` erzeugen jeweils nur nicht gefüllte bzw. nur gefüllte Objekte. Dadurch kann ein Bild nie beide Arten mischen. Die `GraphicManagement` kennt nur die Schnittstelle und die abstrakten Produkte `Circle` und `Rectangle`. Neue Familien (z. B. `HatchedGraphicObjectFactory` mit `HatchedCircle` und `HatchedRectangle`) können ergänzt werden, ohne bestehenden Code zu ändern. Die Fabrik ist pro Bild austauschbar (Parameter von `AddImage`).

Singleton-Pattern Bilderverwaltung und konkrete Fabriken dürfen nur einmal existieren. Dazu dient die Template-Klasse `SingletonBase<T>`: Die Instanz wird beim ersten Aufruf von `GetInstance()` erzeugt und in einem statischen `std::unique_ptr` gehalten. Sie wird dadurch beim Programmende automatisch freigegeben (`Delete()` erlaubt zusätzlich eine vorzeitige Freigabe). Die Konstruktoren der Singletons sind privat bzw. geschützt, die Basisklasse ist dazu als `friend` eingetragen.

Vererbungshierarchie und Kopierverbot Alle Klassen leiten direkt oder indirekt von `Object` ab. Dessen Kopierkonstruktor und Zuweisungsoperator sind gelöscht, womit das Kopieren und Zuweisen aller abgeleiteten Klassen verhindert wird. Der geschützte Konstruktor verhindert die direkte Instanziierung.

Gemeinsame Basisklassen der grafischen Objekte Gemeinsame Attribute (Strichfarbe, Position) liegen in `GraphicObject`, die geometrischen Eigenschaften in `Circle` bzw. `Rectangle`. Die Unterklassen unterscheiden sich nur in `ToSVG()` (Polymorphismus). Die Konstruktoren der Basisklassen sind geschützt, die Objekte werden nur über die Fabriken erzeugt.

Besitzverhältnisse Die Fabriken geben `std::unique_ptr` zurück. Das `Image` besitzt seine Objekte, die `GraphicManagement` besitzt die Bilder und hält die aktuelle Fabrik nur als nicht besitzenden Zeiger (die Fabriken sind selbst Singletons).

Fehlerstrategie Ungültige Aufrufe (keine Fabrik oder kein Bild vorhanden, Datei nicht beschreibbar) werden über Ausnahmen gemeldet und im Testtreiber behandelt.

4 Komponentenentwurf

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

5.1 Ausgabe Konsole Testtreiber

Created blank—cyberpunk.svg and filled—cyberpunk.svg

5.2 Erstellte Bilder

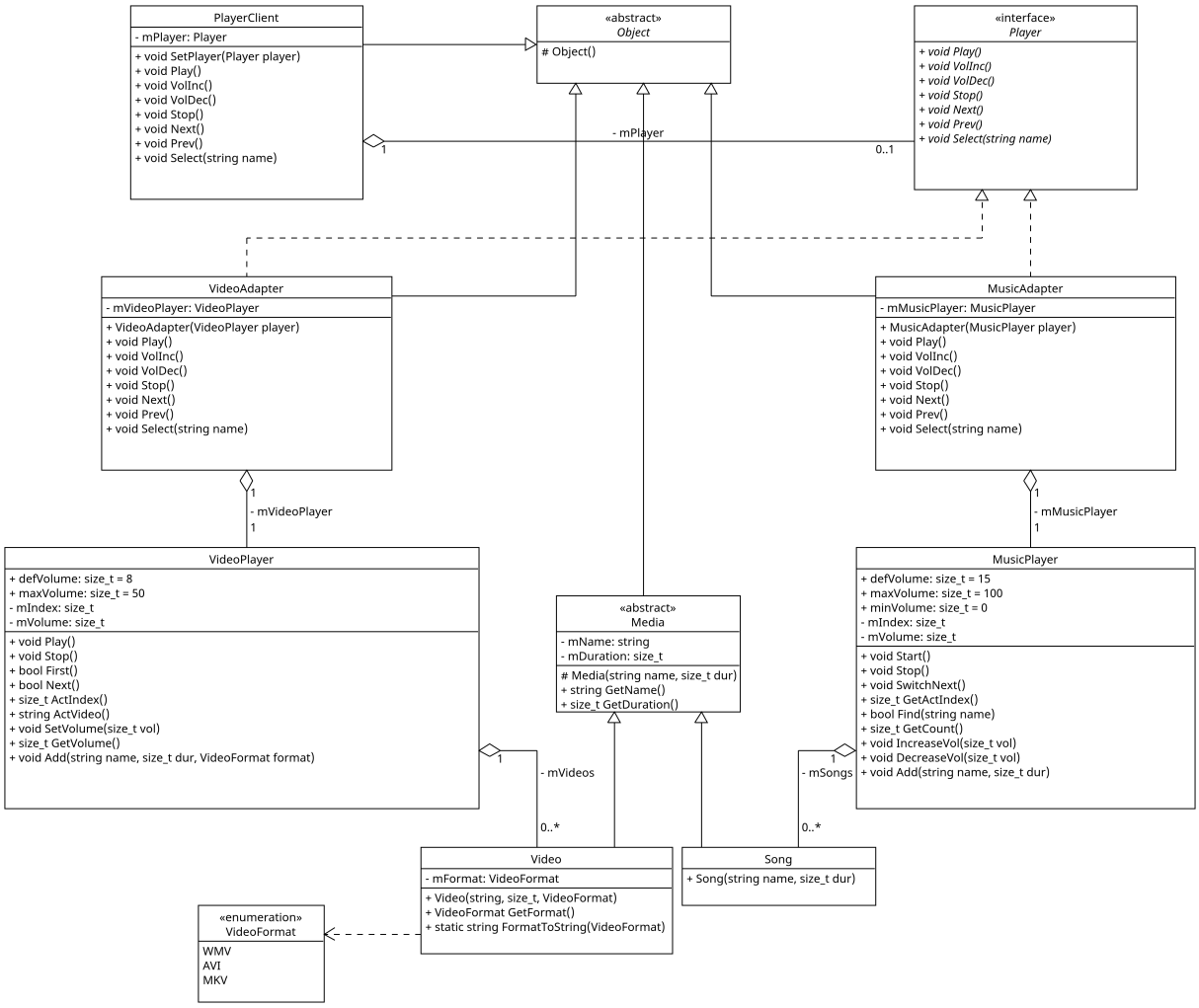


Abbildung 2: Klassendiagramm Bilderverwaltung

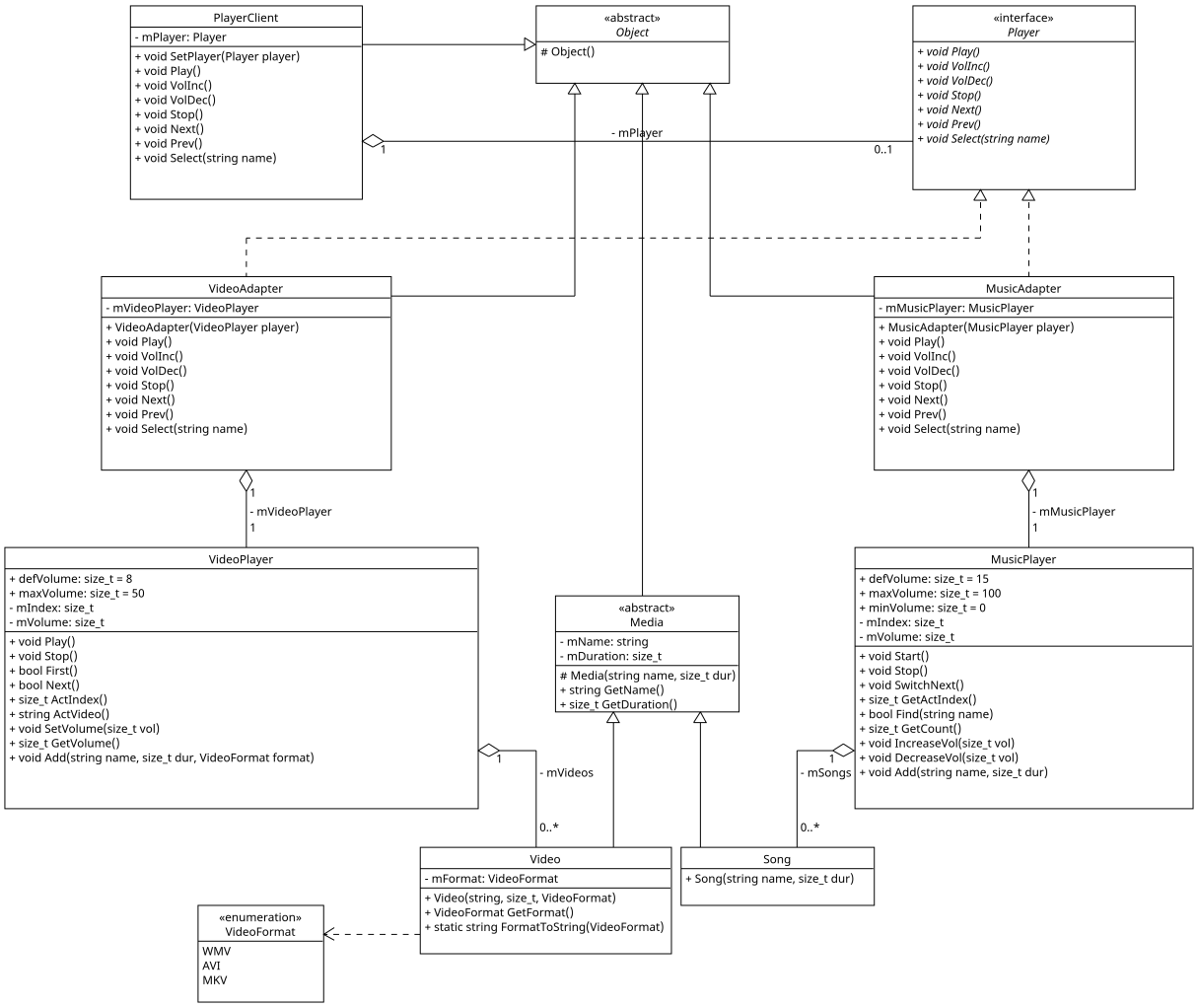


Abbildung 3: Klassendiagramm Bilderverwaltung

6 Quellcode

Listing 1: ../Object.h

```
1  #ifndef OBJECT_H
2  #define OBJECT_H
3
4  /// @author Julian Rieger
5  /// @date 05.11.2026
6  /// @brief Abstract base class of all classes of the image management.
7  ///
8  /// The class can not be instantiated directly (protected constructor) and it
9  /// prevents copying and assigning of all derived classes.
10 class Object
11 {
12 public:
13    /// @brief Copying is not allowed.
14    Object(const Object&) = delete;
15
16    /// @brief Assigning is not allowed.
17    Object& operator=(const Object&) = delete;
18
19    /// @brief Virtual destructor, so derived objects can be deleted via a base
          pointer.
20    virtual ~Object();
21
22 protected:
23    /// @brief Protected constructor, only derived classes can create an Object.
24    Object();
25 };
26
27 #endif
```

Listing 2: ../BlankCircle.h

```
1  #ifndef BLANKCIRCLE_H
2  #define BLANKCIRCLE_H
3  #include <cstddef>
4  #include <string>
5  #include "Circle.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief A not filled circle.
10 ///
11 /// Objects of this class can only be created by the BlankGraphicsObjectFactory.
12 class BlankCircle : public Circle
13 {
14 public:
15    /// @brief Creates the SVG element of this not filled circle.
16    /// @returns the SVG element as text.
17    std::string ToSVG() const override;
18
19    /// @brief Creates the object.
20    /// @param[in] stroke is the color of the outline.
21    /// @param[in] x is the x coordinate.
22    /// @param[in] y is the y coordinate.
23    /// @param[in] radius is the radius of the circle.
24    BlankCircle(const std::string& stroke, int x, int y, size_t radius);
25 };
26
27 #endif
```

Listing 3: ../BlankCircle.cpp

```
1  #include "BlankCircle.h"
2
```

```
3  std::string BlankCircle::ToSVG() const
4  {
5      //<circle cx="55" cy="45" r="35" stroke="black" fill="none"/>
6      std::string svg{};
7      svg += "<circle cx=\"";
8      svg += std::to_string(GetX());
9      svg += "\" cy=\"";
10     svg += std::to_string(GetY());
11     svg += "\" r=\"";
12     svg += std::to_string(GetRadius());
13     svg += "\" stroke=\"";
14     svg += GetStroke();
15     svg += "\" fill=\"none\"/>";
16     return svg;
17 }
18
19 BlankCircle::BlankCircle(const std::string& stroke, int x, int y, size_t radius) :
20     Circle{stroke, x, y, radius}
21 {
22 }
```

Listing 4: ../FillCircle.h

```
1  #ifndef FILLEDCIRCLE_H
2  #define FILLEDCIRCLE_H
3  #include <cstddef>
4  #include <string>
5  #include "Circle.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief A filled circle.
10 ///
11 /// Objects of this class can only be created by the FillGraphicsObjectFactory.
12 class FillCircle : public Circle
13 {
14 public:
15    /// @brief Creates the SVG element of this filled circle.
16    /// @returns the SVG element as text.
17    std::string ToSVG() const override;
18
19    /// @brief Creates the object.
20    /// @param[in] stroke is the color of the outline (and of the fill).
21    /// @param[in] x is the x coordinate.
22    /// @param[in] y is the y coordinate.
23    /// @param[in] radius is the radius of the circle.
24    FillCircle(const std::string& stroke, int x, int y, size_t radius);
25 };
26
27 #endif
```

Listing 5: ../FillCircle.cpp

```
1  #include "FillCircle.h"
2
3  FillCircle::FillCircle(const std::string& stroke, int x, int y, size_t radius)
4      : Circle(stroke, x, y, radius)
5  {
6  }
7
8  std::string FillCircle::ToSVG() const
9  {
10     //<circle cx="55" cy="45" r="35" stroke="black"/>
11     std::string svg{};
12     svg += "<circle cx=\"";
13     svg += std::to_string(GetX());
14     svg += "\" cy=\"";
```

```
15      svg += std::to_string(GetY());
16      svg += "\" r=\"";
17      svg += std::to_string(GetRadius());
18      svg += "\" stroke=\"";
19      svg += GetStroke();
20      svg += "\" fill=\"";
21      svg += GetStroke();
22      svg += "\"/>";
23      return svg;
24  }
```

Listing 6: ../Circle.h

```
1   #ifndef CIRCLE_H
2   #define CIRCLE_H
3   #include <cstddef>
4   #include <string>
5   #include "GraphicObject.h"
6
7   /// @author Julian Rieger
8   /// @date 05.11.2026
9   /// @brief Abstract base class of all circles.
10  ///
11  /// The position (mX, mY) is the center of the circle. Whether the circle is
12  /// filled or not is decided by the derived classes in ToSVG.
13  class Circle : public GraphicObject
14  {
15  public:
16      /// @brief Creates the SVG element of the circle, implemented by the derived
            classes.
17      /// @returns the SVG element as text.
18      std::string ToSVG() const override = 0;
19
20  protected:
21      /// @brief Creates a circle.
22      /// @param[in] stroke is the color of the outline.
23      /// @param[in] x is the x coordinate of the center.
24      /// @param[in] y is the y coordinate of the center.
25      /// @param[in] radius is the radius of the circle.
26      /// @throws std::invalid_argument if radius is 0.
27      Circle(const std::string& stroke, int x, int y, size_t radius);
28
29      /// @brief Access function for the radius.
30      /// @returns the radius.
31      size_t GetRadius() const;
32
33  private:
34      /// @brief Radius of the circle.
35      size_t mRadius;
36  };
37
38  #endif
```

Listing 7: ../Circle.cpp

```
1   #include "Circle.h"
2   #include <stdexcept>
3
4   Circle::Circle(const std::string& stroke, int x, int y, size_t radius) :
5       GraphicObject{stroke, x, y},
6       mRadius{radius}
7   {
8       if (radius == 0) {
9           throw std::invalid_argument("Radius must not be 0");
10      }
11  }
12
```

```
13 size_t Circle::GetRadius() const
14 {
15     return mRadius;
16 }
```

Listing 8: ../BlankRectangle.h

```
1  #ifndef BLANKRECTANGLE_H
2  #define BLANKRECTANGLE_H
3  #include <cstddef>
4  #include <string>
5  #include "Rectangle.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief A not filled rectangle.
10 ///
11 /// Objects of this class can only be created by the BlankGraphicsObjectFactory.
12 class BlankRectangle : public Rectangle
13 {
14 public:
15    /// @brief Creates the SVG element of this not filled rectangle.
16    /// @returns the SVG element as text.
17    std::string ToSVG() const override;
18
19    /// @brief Creates the object.
20    /// @param[in] stroke is the color of the outline.
21    /// @param[in] x is the x coordinate.
22    /// @param[in] y is the y coordinate.
23    /// @param[in] width is the width of the rectangle.
24    /// @param[in] height is the height of the rectangle.
25    BlankRectangle(const std::string& stroke, int x, int y, size_t width, size_t
          height);
26 };
27
28 #endif
```

Listing 9: ../BlankRectangle.cpp

```
1  #include "BlankRectangle.h"
2
3  BlankRectangle::BlankRectangle(const std::string& stroke, int x, int y, size_t
       width, size_t height) :
4      Rectangle{stroke, x, y, width, height}
5  {
6  }
7
8  std::string BlankRectangle::ToSVG() const
9  {
10     //<rect x="10" y="10" width="90" height="70" stroke="black" fill="none"/>
11     std::string svg{};
12     svg += "<rect x=\"";
13     svg += std::to_string(GetX());
14     svg += "\" y=\"";
15     svg += std::to_string(GetY());
16     svg += "\" width=\"";
17     svg += std::to_string(GetWidth());
18     svg += "\" height=\"";
19     svg += std::to_string(GetHeight());
20     svg += "\" stroke=\"";
21     svg += GetStroke();
22     svg += "\" fill=\"none\"/>";
23     return svg;
24 }
```

Listing 10: ../FillRectangle.h

```h
#ifndef FILLRECTANGLE_H
#define FILLRECTANGLE_H
#include <cstddef>
#include <string>
#include "Rectangle.h"

/// @author Julian Rieger
/// @date 05.11.2026
/// @brief A filled rectangle.
///
/// Objects of this class can only be created by the FillGraphicsObjectFactory.
class FillRectangle : public Rectangle
{
public:
    /// @brief Creates the SVG element of this filled rectangle.
    /// @returns the SVG element as text.
    std::string ToSVG() const override;

    /// @brief Creates the object.
    /// @param[in] stroke is the color of the outline (and of the fill).
    /// @param[in] x is the x coordinate.
    /// @param[in] y is the y coordinate.
    /// @param[in] width is the width of the rectangle.
    /// @param[in] height is the height of the rectangle.
    FillRectangle(const std::string& stroke, int x, int y, size_t width, size_t
        height);
};

#endif
```

Listing 11: ../FillRectangle.cpp

```cpp
#include "FillRectangle.h"

FillRectangle::FillRectangle(const std::string& stroke, int x, int y, size_t width
    , size_t height) :
    Rectangle{stroke, x, y, width, height}
{
}

std::string FillRectangle::ToSVG() const
{
    //<rect x="10" y="10" width="90" height="70" stroke="black"/>
    std::string svg{};
    svg += "<rect x=\"";
    svg += std::to_string(GetX());
    svg += "\" y=\"";
    svg += std::to_string(GetY());
    svg += "\" width=\"";
    svg += std::to_string(GetWidth());
    svg += "\" height=\"";
    svg += std::to_string(GetHeight());
    svg += "\" stroke=\"";
    svg += GetStroke();
    svg += "\" fill=\"";
    svg += GetStroke();
    svg += "\"/>";
    return svg;
}
```

Listing 12: ../Rectangle.h

```h
#ifndef RECTANGLE_H
#define RECTANGLE_H
#include <cstddef>
#include <string>
```

```
5  #include "GraphicObject.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief Abstract base class of all rectangles.
10 ///
11 /// The position (mX, mY) is the upper left corner of the rectangle. Whether
12 /// the rectangle is filled or not is decided by the derived classes in ToSVG.
13 class Rectangle : public GraphicObject
14 {
15 public:
16     /// @brief Creates the SVG element of the rectangle, implemented by the derived
           classes.
17     /// @returns the SVG element as text.
18     std::string ToSVG() const override = 0;
19
20 protected:
21     /// @brief Creates a rectangle.
22     /// @param[in] stroke is the color of the outline.
23     /// @param[in] x is the x coordinate of the upper left corner.
24     /// @param[in] y is the y coordinate of the upper left corner.
25     /// @param[in] width is the width of the rectangle.
26     /// @param[in] height is the height of the rectangle.
27     /// @throws std::invalid_argument if width or height is 0.
28     Rectangle(const std::string& stroke, int x, int y, size_t width, size_t height)
           ;
29
30     /// @brief Access function for the width.
31     /// @returns the width.
32     size_t GetWidth() const;
33
34     /// @brief Access function for the height.
35     /// @returns the height.
36     size_t GetHeight() const;
37
38 private:
39     /// @brief Width of the rectangle.
40     size_t mWidth;
41
42     /// @brief Height of the rectangle.
43     size_t mHeight;
44 };
45
46 #endif
```

Listing 13: ../Rectangle.cpp

```
1  #include "Rectangle.h"
2  #include <stdexcept>
3
4  Rectangle::Rectangle(const std::string& stroke, int x, int y, size_t width, size_t
       height) :
5      GraphicObject{stroke, x, y},
6      mWidth{width},
7      mHeight{height}
8  {
9      if (width == 0 || height == 0) {
10         throw std::invalid_argument("Width and height must not be 0");
11     }
12 }
13
14 size_t Rectangle::GetWidth() const
15 {
16     return mWidth;
17 }
18
19 size_t Rectangle::GetHeight() const
20 {
21     return mHeight;
```

```
22 }
```

Listing 14: ../SingletonBase.h

```
1  #ifndef SINGLETONBASE_H
2  #define SINGLETONBASE_H
3
4  #include <memory>
5  #include "Object.h"
6
7  /// @author Leander Kieweg
8  /// @date 05.11.2026
9  /// @brief Template base class for the Singleton design pattern.
10 ///
11 /// Ensures that only one instance of the derived class T exists.
12 template <typename T>
13 class SingletonBase : public Object
14 {
15 public:
16     /// @brief Accesses the single instance of the class.
17     /// Creates the instance if it does not exist yet.
18     /// @returns Reference to the singleton instance.
19     static T& GetInstance()
20     {
21         if (mInstance == nullptr)
22         {
23             mInstance = std::unique_ptr<T>{ new T{} };
24         }
25
26         return *mInstance;
27     }
28
29     /// @brief Frees the singleton instance before the end of the program.
30     static void Delete()
31     {
32         mInstance.reset();
33     }
34
35 protected:
36     /// @brief Protected constructor so that only derived classes can be created.
37     SingletonBase() = default;
38
39     /// @brief Virtual destructor.
40     virtual ~SingletonBase() = default;
41
42 private:
43     SingletonBase(SingletonBase const& s) = delete;
44     SingletonBase& operator=(SingletonBase const& s) = delete;
45
46     /// @brief Pointer to the single instance.
47     static std::unique_ptr<T> mInstance;
48 };
49
50 // Initialize static member
51 template <typename T>
52 std::unique_ptr<T> SingletonBase<T>::mInstance = nullptr;
53
54 #endif
```

Listing 15: ../IGraphicObjectFactory.h

```
1 #ifndef IGRAPHICOBJECTFACTORY_H
2 #define IGRAPHICOBJECTFACTORY_H
3 #include <cstddef>
4 #include <memory>
5 #include <string>
6 #include "Circle.h"
```

```
7  #include "Rectangle.h"
8
9  /// @author Leander Kieweg
10 /// @date 05.11.2026
11 /// @brief Interface of the abstract factory which creates graphical objects.
12 ///
13 /// Every concrete factory creates a family of objects (e.g. only filled or
14 /// only not filled ones). New families (hatched, dotted, ...) can be added by
15 /// implementing this interface, without changing the existing classes.
16 class IGraphicObjectFactory
17 {
18 public:
19     /// @brief Virtual destructor.
20     virtual ~IGraphicObjectFactory() = default;
21
22     /// @brief Creates a circle.
23     /// @param[in] stroke is the color of the outline.
24     /// @param[in] x is the x coordinate of the center.
25     /// @param[in] y is the y coordinate of the center.
26     /// @param[in] radius is the radius of the circle.
27     /// @returns the created circle, the caller owns it.
28     virtual std::unique_ptr<Circle> CreateCircle(const std::string& stroke, int x,
           int y, size_t radius) const = 0;
29
30     /// @brief Creates a rectangle.
31     /// @param[in] stroke is the color of the outline.
32     /// @param[in] x is the x coordinate of the upper left corner.
33     /// @param[in] y is the y coordinate of the upper left corner.
34     /// @param[in] width is the width of the rectangle.
35     /// @param[in] height is the height of the rectangle.
36     /// @returns the created rectangle, the caller owns it.
37     virtual std::unique_ptr<Rectangle> CreateRectangle(const std::string& stroke,
           int x, int y, size_t width, size_t height) const = 0;
38 };
39
40 #endif
```

Listing 16: ../BlankGraphicObjectFactory.h

```
1  #ifndef BLANKGRAPHICOBJECTFACTORY_H
2  #define BLANKGRAPHICOBJECTFACTORY_H
3  #include <cstddef>
4  #include <memory>
5  #include <string>
6  #include "IGraphicObjectFactory.h"
7  #include "Object.h"
8  #include "SingletonBase.h"
9
10 /// @author Leander Kieweg
11 /// @date 05.11.2026
12 /// @brief Factory which creates not filled circles and rectangles.
13 ///
14 /// The factory exists only once (singleton). It is created at the first call
15 /// of GetInstance and released automatically when the program ends.
16 class BlankGraphicObjectFactory : public IGraphicObjectFactory, public
       SingletonBase<BlankGraphicObjectFactory>
17 {
18     friend class SingletonBase<BlankGraphicObjectFactory>;
19 public:
20     /// @brief Creates a not filled circle.
21     /// @param[in] stroke is the color of the outline.
22     /// @param[in] x is the x coordinate of the center.
23     /// @param[in] y is the y coordinate of the center.
24     /// @param[in] radius is the radius of the circle.
25     /// @returns the created circle, the caller owns it.
26     std::unique_ptr<Circle> CreateCircle(const std::string& stroke, int x, int y,
           size_t radius) const override;
27
28     /// @brief Creates a not filled rectangle.
```

```
29     /// @param[in] stroke is the color of the outline.
30     /// @param[in] x is the x coordinate of the upper left corner.
31     /// @param[in] y is the y coordinate of the upper left corner.
32     /// @param[in] width is the width of the rectangle.
33     /// @param[in] height is the height of the rectangle.
34     /// @returns the created rectangle, the caller owns it.
35     std::unique_ptr<Rectangle> CreateRectangle(const std::string& stroke, int x,
           int y, size_t width, size_t height) const override;
36
37 private:
38     /// @brief Private constructor, use GetInstance.
39     BlankGraphicObjectFactory() = default;
40 };
41
42 #endif
```

Listing 17: ../BlankGraphicObjectFactory.cpp

```
1  #include "BlankGraphicObjectFactory.h"
2  #include "BlankCircle.h"
3  #include "BlankRectangle.h"
4
5  std::unique_ptr<Circle> BlankGraphicObjectFactory::CreateCircle(const std::string
       &stroke, int x, int y,
6                                                                  size_t radius)
                                                                       const {
7      return std::make_unique<BlankCircle>(stroke, x, y, radius);
8  }
9
10 std::unique_ptr<Rectangle> BlankGraphicObjectFactory::CreateRectangle(const std::
       string &stroke, int x, int y,
11     size_t width, size_t height) const {
12     return std::make_unique<BlankRectangle>(stroke, x, y, width, height);
13 }
```

Listing 18: ../FillGraphicObjectFactory.h

```
1  #ifndef FILLGRAPHICOBJECTFACTORY_H
2  #define FILLGRAPHICOBJECTFACTORY_H
3  #include <cstddef>
4  #include <memory>
5  #include <string>
6  #include "IGraphicObjectFactory.h"
7  #include "Object.h"
8  #include "SingletonBase.h"
9
10 /// @author Leander Kieweg
11 /// @date 05.11.2026
12 /// @brief Factory which creates filled circles and rectangles.
13 ///
14 /// The factory exists only once (singleton). It is created at the first call
15 /// of GetInstance and released automatically when the program ends.
16 class FillGraphicObjectFactory : public IGraphicObjectFactory, public
       SingletonBase<FillGraphicObjectFactory>
17 {
18     friend class SingletonBase<FillGraphicObjectFactory>;
19 public:
20
21     /// @brief Creates a filled circle.
22     /// @param[in] stroke is the color of the outline.
23     /// @param[in] x is the x coordinate of the center.
24     /// @param[in] y is the y coordinate of the center.
25     /// @param[in] radius is the radius of the circle.
26     /// @returns the created circle, the caller owns it.
27     std::unique_ptr<Circle> CreateCircle(const std::string& stroke, int x, int y,
           size_t radius) const override;
28
```

```
29     /// @brief Creates a filled rectangle.
30     /// @param[in] stroke is the color of the outline.
31     /// @param[in] x is the x coordinate of the upper left corner.
32     /// @param[in] y is the y coordinate of the upper left corner.
33     /// @param[in] width is the width of the rectangle.
34     /// @param[in] height is the height of the rectangle.
35     /// @returns the created rectangle, the caller owns it.
36     std::unique_ptr<Rectangle> CreateRectangle(const std::string& stroke, int x,
           int y, size_t width, size_t height) const override;
37
38 private:
39     /// @brief Private constructor, use GetInstance.
40     FillGraphicObjectFactory() = default;
41 };
42
43 #endif
```

Listing 19: ../FillGraphicObjectFactory.cpp

```
1  #include "FillGraphicObjectFactory.h"
2  #include "FillCircle.h"
3  #include "FillRectangle.h"
4
5  std::unique_ptr<Circle> FillGraphicObjectFactory::CreateCircle(const std::string &
       stroke, int x, int y,
6                                                                 size_t radius)
                                                                      const {
7      return std::make_unique<FillCircle>(stroke, x, y, radius);
8  }
9
10 std::unique_ptr<Rectangle> FillGraphicObjectFactory::CreateRectangle(const std::
       string &stroke, int x, int y,
11     size_t width, size_t height) const {
12     return std::make_unique<FillRectangle>(stroke, x, y, width, height);
13 }
```

Listing 20: ../Image.h

```
1  #ifndef IMAGE_H
2  #define IMAGE_H
3  #include <memory>
4  #include <string>
5  #include <vector>
6  #include <fstream>
7  #include "GraphicObject.h"
8  #include "Object.h"
9
10 /// @author Julian Rieger
11 /// @date 05.11.2026
12 /// @brief An image which consists of graphical objects and is stored as SVG file.
13 ///
14 class Image : public Object
15 {
16 public:
17     /// @brief Creates an image and its graphical objects.
18     /// @param[in] filename is the name of the SVG file.
19     explicit Image(const std::string& filename);
20
21     /// @brief Writes the image as SVG file.
22     /// @throws std::runtime_error if the file can not be opened.
23     void WriteSVG() const;
24
25     /// @brief Adds a graphic object to the vector.
26     /// Uses push_back() internally.
27     void AddGraphicObject(std::unique_ptr<GraphicObject> obj);
28
29 private:
```

```
30
31      /// @brief Name of the SVG file.
32      std::string mFilename;
33
34      /// @brief All graphical objects of the image, owned by the image.
35      std::vector<std::unique_ptr<GraphicObject>> mGraphicObjects;
36
37      /// @brief Prints the formatted header of the SVG file
38      void PrintHeader(std::ostream& ost) const;
39
40      /// @brief Prints the formatted footer of the SVG file
41      void PrintFooter(std::ostream& ost) const;
42
43  };
44
45  #endif
```

Listing 21: ../Image.cpp

```
1   #include "Image.h"
2   #include <fstream>
3   #include <stdexcept>
4   #include <utility>
5
6   Image::Image(const std::string& filename) :
7       mFilename{ filename }
8   {
9   }
10
11  void Image::WriteSVG() const
12  {
13      std::ofstream file(mFilename);
14      if (!file.is_open())
15          throw std::runtime_error("Error: Can not open file " + mFilename);
16
17      PrintHeader(file);
18      for (const auto& elem : mGraphicObjects)
19          file << elem->ToSVG() << "\n";
20      PrintFooter(file);
21
22      file.close();
23  }
24
25  void Image::AddGraphicObject(std::unique_ptr<GraphicObject> obj)
26  {
27      mGraphicObjects.push_back(std::move(obj));
28  }
29
30  void Image::PrintHeader(std::ostream& out) const
31  {
32      out << "<?xml version=\"1.0\"?>\n"
33          << "<svg xmlns=\"http://www.w3.org/2000/svg\">\n";
34  }
35
36  void Image::PrintFooter(std::ostream& out) const
37  {
38      out << "</svg>\n";
39  }
```

Listing 22: ../GraphicManagement.h

```
1   #ifndef GRAPHICMANAGEMENT_H
2   #define GRAPHICMANAGEMENT_H
3   #include <cstddef>
4   #include <memory>
5   #include <string>
6   #include <vector>
```

```
7  #include "Circle.h"
8  #include "IGraphicObjectFactory.h"
9  #include "Object.h"
10 #include "Rectangle.h"
11 #include "SingletonBase.h"
12
13 class Image;
14
15 /// @author Leander Kieweg
16 /// @date 05.11.2026
17 /// @brief Manages all images (singleton).
18 ///
19 /// The management exists only once and is released automatically when the
20 /// program ends. The factory that creates the graphical objects can be
21 /// exchanged: each call of AddImage sets the factory for the new image.
22 class GraphicManagement : public SingletonBase<GraphicManagement>
23 {
24    friend class SingletonBase<GraphicManagement>;
25 public:
26    ~GraphicManagement() override;
27
28    /// @brief Creates a new image with the given factory and stores it.
29    /// @param[in] fact is the factory used for all objects of the new image.
30    /// @param[in] file is the name of the SVG file of the image.
31    void AddImage(const IGraphicObjectFactory& fact, const std::string& file);
32
33    /// @brief Writes all images as SVG files.
34    /// @throws std::runtime_error if a file can not be opened.
35    void WriteSVG() const;
36
37    /// @brief Creates a circle with the current factory.
38    /// @param[in] stroke is the color of the outline.
39    /// @param[in] x is the x coordinate of the center.
40    /// @param[in] y is the y coordinate of the center.
41    /// @param[in] radius is the radius of the circle.
42    /// @returns the created circle, the caller owns it.
43    /// @throws std::logic_error if no factory is set.
44    void CreateCircle(const std::string& stroke, int x, int y, size_t radius);
45
46    /// @brief Creates a rectangle with the current factory.
47    /// @param[in] stroke is the color of the outline.
48    /// @param[in] x is the x coordinate of the upper left corner.
49    /// @param[in] y is the y coordinate of the upper left corner.
50    /// @param[in] width is the width of the rectangle.
51    /// @param[in] height is the height of the rectangle.
52    /// @returns the created rectangle, the caller owns it.
53    /// @throws std::logic_error if no factory is set.
54    void CreateRectangle(const std::string& stroke, int x, int y, size_t width,
        size_t height);
55
56 private:
57    /// @brief Private constructor, use GetInstance.
58    GraphicManagement();
59
60    /// @brief Factory for the next images, not owned (the factories are singletons
       ).
61    const IGraphicObjectFactory* mFactory = nullptr;
62
63    /// @brief All images, owned by the management.
64    std::vector<std::unique_ptr<Image>> mImages;
65 };
66
67 #endif
```

Listing 23: ../GraphicManagement.cpp

```
1 #include "GraphicManagement.h"
2 #include "Image.h"
3 #include <stdexcept>
```

```
4
5  GraphicManagement::~GraphicManagement() = default;
6  GraphicManagement::GraphicManagement() = default;
7
8  void GraphicManagement::AddImage(const IGraphicObjectFactory &fact, const std::
       string &file) {
9      mFactory = &fact;
10     mImages.push_back(std::make_unique<Image>(file));
11 }
12
13 void GraphicManagement::WriteSVG() const {
14     for (const auto &image : mImages) {
15         image->WriteSVG();
16     }
17 }
18
19 void GraphicManagement::CreateCircle(const std::string &stroke, int x, int y,
       size_t radius) {
20     if (mFactory == nullptr) {
21         throw std::logic_error("No factory is set.");
22     }
23
24     if (mImages.empty()) {
25         throw std::logic_error("No image has been added.");
26     }
27
28     mImages.back()->AddGraphicObject(
29         mFactory->CreateCircle(stroke, x, y, radius));
30 }
31
32 void GraphicManagement::CreateRectangle(const std::string &stroke, int x, int y,
       size_t width,
33                                         size_t height) {
34     if (mFactory == nullptr) {
35         throw std::logic_error("No factory is set.");
36     }
37
38     if (mImages.empty()) {
39         throw std::logic_error("No image has been added.");
40     }
41
42     mImages.back()->AddGraphicObject(
43         mFactory->CreateRectangle(stroke, x, y, width, height));
44 }
```

Listing 24: ../main.cpp

```
1  #include <cstdint>
2  #include <cstdio>
3  #include <exception>
4  #include <iostream>
5  <<<<<<< HEAD
6  #include <string>
7  =======
8  #include <stdexcept>
9  #include <new>
10 >>>>>>> dc61e758c33d18c066571b562efcead22a9554c5
11 #include "BlankGraphicObjectFactory.h"
12 #include "FillGraphicObjectFactory.h"
13 #include "GraphicManagement.h"
14
15 namespace
16 {
17     /// @brief Simple deterministic random number generator (LCG).
18     ///
19     /// A fixed seed is used on purpose: the blank and the filled image are
20     /// painted by two separate calls and must show exactly the same scene.
21     class Random
22     {
```

```
23      public:
24          explicit Random(std::uint32_t seed) : mState{ seed } {}
25
26          /// @returns a number in the range [lo, hi].
27          int Next(int lo, int hi)
28          {
29              mState = mState * 1664525u + 1013904223u;
30              return lo + static_cast<int>((mState >> 8) % static_cast<std::uint32_t
                    >(hi - lo + 1));
31          }
32
33      private:
34          std::uint32_t mState;
35      };
36
37      /// @returns the color as "#rrggbb" string.
38      std::string Hex(int r, int g, int b)
39      {
40          char buf[8];
41          std::snprintf(buf, sizeof(buf), "#%02x%02x%02x", r, g, b);
42          return buf;
43      }
44
45      /// @returns the linear interpolation of a and b (t in 0..100).
46      int Mix(int a, int b, int t)
47      {
48          return a + (b - a) * t / 100;
49      }
50  } // namespace
51
52  /// @brief Paints a night scene (800 x 600) with far more than 500 shapes.
53  ///
54  /// Layers (painter's algorithm, later objects cover earlier ones):
55  ///   1. sky as 30 horizontal color bands (gradient)    ->  30 shapes
56  ///   2. 220 stars                                       -> 220 shapes
57  ///   3. moon with halo and craters                      ->  12 shapes
58  ///   4. mountain range                                  ->  15 shapes
59  ///   5. 26 houses with roofs, chimneys and windows      -> ~600 shapes
60  ///   6. street with lamps and their light cones         ->  ~60 shapes
61  static void PaintNightCity(GraphicManagement& gm)
62  {
63      Random rnd{ 2026 };
64
65      // ---------- 1. Sky gradient: deep blue on top, orange at the horizon
            ----------
66      for (int i = 0; i < 30; ++i) {
67          int t = i * 100 / 29;
68          gm.CreateRectangle(Hex(Mix(8, 232, t * t / 100), Mix(10, 120, t * t / 100)
                , Mix(48, 90, t)),
69              0, i * 14, 800, 14);
70      }
71
72      // ---------- 2. Stars (more and bigger ones in the upper sky) ----------
73      for (int i = 0; i < 220; ++i) {
74          int x = rnd.Next(0, 800);
75          int y = rnd.Next(0, 330);
76          int size = rnd.Next(0, 9) == 0 ? 2 : 1;
77          int bright = rnd.Next(170, 255);
78          gm.CreateCircle(Hex(bright, bright, rnd.Next(190, 255)), x, y, size);
79      }
80
81      // ---------- 3. Moon ----------
82      gm.CreateCircle("#2a3a78", 630, 110, 78);   // outer halo
83      gm.CreateCircle("#3d4f95", 630, 110, 62);
84      gm.CreateCircle("#f5f0c8", 630, 110, 46);   // moon disc
85      gm.CreateCircle("#e3dcae", 612, 96, 11);    // craters
86      gm.CreateCircle("#e3dcae", 650, 124, 9);
87      gm.CreateCircle("#e3dcae", 640, 84, 6);
88      gm.CreateCircle("#e3dcae", 618, 130, 7);
89      gm.CreateCircle("#d8d09a", 660, 100, 5);
```

```
90      gm.CreateCircle("#d8d09a", 600, 116, 4);
91      gm.CreateCircle("#fffbe0", 598, 90, 3);      // highlights
92      gm.CreateCircle("#fffbe0", 660, 140, 3);
93
94      // ---------- 4. Mountains: overlapping circles, two rows ----------
95      for (int i = 0; i < 9; ++i)
96          gm.CreateCircle("#3a2a5c", i * 100 - 20, 440, rnd.Next(70, 120));   // far
                row
97      for (int i = 0; i < 6; ++i)
98          gm.CreateCircle("#241a40", i * 150 + 30, 470, rnd.Next(60, 95));    //
                near row
99      gm.CreateRectangle("#1b1432", 0, 440, 800, 160);                         //
            ground below mountains
100
101     // ---------- 5. Houses ----------
102     // 26 houses of width 30 side by side, the bottom edge is y = 520.
103     for (int h = 0; h < 26; ++h) {
104         int x = h * 31;
105         int width = 30;
106         int height = rnd.Next(90, 250);
107         int top = 520 - height;
108         int shade = rnd.Next(0, 24);
109
110         gm.CreateRectangle(Hex(34 + shade, 28 + shade, 66 + shade), x, top, width,
                height);  // wall
111         gm.CreateRectangle(Hex(60 + shade, 40 + shade, 90 + shade), x - 1, top -
                5, width + 2, 6); // roof edge
112         if (rnd.Next(0, 2) == 0)
113             gm.CreateRectangle("#120d26", x + 20, top - 16, 5, 12);
                                            // chimney
114
115         // windows: 2 columns, a new row every 14 pixels
116         for (int wy = top + 10; wy < 512; wy += 14) {
117             for (int wx = x + 6; wx < x + width - 6; wx += 12) {
118                 bool lit = rnd.Next(0, 99) < 62;
119                 std::string color = lit ? Hex(255, rnd.Next(200, 235), rnd.Next
                        (80, 140))
120                     : Hex(18, 16, 40);
121                 gm.CreateRectangle(color, wx, wy, 7, 8);
122             }
123         }
124     }
125
126     // ---------- 6. Street, lamps and puddles ----------
127     gm.CreateRectangle("#2b2342", 0, 520, 800, 80);          // street
128     gm.CreateRectangle("#4a3f6b", 0, 520, 800, 4);           // curb
129     for (int i = 0; i < 8; ++i) {
130         int x = 50 + i * 100;
131         gm.CreateRectangle("#0e0a1e", x, 470, 4, 75);        // lamp post
132         gm.CreateCircle("#fff2a8", x + 2, 466, 5);           // lamp
133         gm.CreateCircle("#c9b25a", x + 2, 530, 24);          // light on the
                street
134         gm.CreateCircle("#ffe98a", x + 2, 530, 14);
135     }
136     for (int i = 0; i < 12; ++i)                             // road markings
137         gm.CreateRectangle("#8e86ad", i * 70 + 10, 565, 36, 4);
138 }
139
140 /// @author Leander Kieweg
141 /// @date 05.11.2026
142
143 /// @brief Test driver: creates images and tests functionality/error handling.
144 int main()
145 {
146     // ========================================
147     // TEST 1: Happy Path & Regular Functionality
148     // ========================================
149     try {
150         auto& gm = GraphicManagement::GetInstance();
151
```

```
152         gm.AddImage(BlankGraphicObjectFactory::GetInstance(), "blank-nightcity.svg
                ");
153         PaintNightCity(gm);
154 
155         gm.AddImage(FillGraphicObjectFactory::GetInstance(), "filled-nightcity.svg
                ");
156         PaintNightCity(gm);
157 
158         gm.WriteSVG();
159     }
160     catch (const std::exception& e) {
161         std::cerr << "Error: " << e.what() << std::endl;
162         return 1;
163     }
164 
165     std::cout << "Created blank-nightcity.svg and filled-nightcity.svg\n";
166         // Outline version: a neon portal above a futuristic skyline.
167         gm.AddImage(BlankGraphicObjectFactory::GetInstance(), "blank-cyberpunk.svg
                ");
168         gm.CreateRectangle("#09001f", 0, 0, 600, 400);
169         gm.CreateCircle("#ff2bd6", 300, 135, 120);
170 
171         // ---> HIER WIEDER DEINE RESTLICHEN KREISE & RECHTECKE EINFÃœGEN <---
172 
173         // Filled version: the same scene with intense neon colors.
174         gm.AddImage(FillGraphicObjectFactory::GetInstance(), "filled-cyberpunk.svg
                ");
175         gm.CreateRectangle("#09001f", 0, 0, 600, 400);
176         gm.CreateCircle("#ff2bd6", 300, 135, 120);
177 
178         // ---> HIER WIEDER DEINE RESTLICHEN KREISE & RECHTECKE EINFÃœGEN <---
179 
180 
181         // --- Checklist Test Cases (1 element & empty container) ---
182 
183         // Test: Container with exactly 1 element
184         gm.AddImage(FillGraphicObjectFactory::GetInstance(), "test-1-element.svg")
                ;
185         gm.CreateCircle("#ff0000", 100, 100, 50);
186 
187         // Test: Empty container (0 elements)
188         gm.AddImage(BlankGraphicObjectFactory::GetInstance(), "test-empty-image.
                svg");
189         // Intentionally not adding any objects here!
190 
191         gm.WriteSVG();
192         std::cout << "[SUCCESS] Images generated successfully.\n";
193 
194     } catch (const std::bad_alloc& e) {
195         std::cerr << "[FATAL ERROR] Memory allocation failed: " << e.what() << "\n
                ";
196         return 1;
197     } catch (const std::exception &e) {
198         std::cerr << "[ERROR] " << e.what() << "\n";
199         return 1;
200     }
201 
202     // ========================================
203     // TEST 2: Provoke and catch errors
204     // ========================================
205 
206     // Test: Check parameter validity (Checklist point 21)
207     try {
208         auto& gm = GraphicManagement::GetInstance();
209         std::cout << "Test: Attempting to create a circle with radius 0...\n";
210         gm.CreateCircle("#000000", 10, 10, 0); // Should throw an exception
211         std::cerr << "[FAIL] No exception thrown for radius 0!\n";
212     } catch (const std::invalid_argument& e) {
213         std::cout << "[EXPECTED] Exception caught: " << e.what() << "\n";
214     }
```

```
215
216     // Test: Empty/Invalid file path (Checklist point 21)
217     try {
218         auto& gm = GraphicManagement::GetInstance();
219         std::cout << "Test: Attempting to save to an invalid path...\n";
220         // An illegal path that should fail on any OS
221         gm.AddImage(BlankGraphicObjectFactory::GetInstance(), "Y:/this/directory/
                does/not/exist/test.svg");
222         gm.WriteSVG();
223         std::cerr << "[FAIL] File could be saved to an invalid path!\n";
224     } catch (const std::runtime_error& e) {
225         std::cout << "[EXPECTED] Exception caught: " << e.what() << "\n";
226     } catch (const std::exception& e) {
227         std::cout << "[EXPECTED] Exception caught: " << e.what() << "\n";
228     }
229
230     std::cout << "\nAll tests completed.\n";
231     return 0;
232 }
```